```
import warnings
warnings.filterwarnings("ignore")
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load Iris dataset
iris = load_iris()

# Create DataFrame
data = pd.DataFrame(iris.data, columns=iris.feature_names)

# Display first 5 rows
data.head()

# Check dataset shape
print("Dataset Shape:", data.shape)

# Display dataset information
data.info()

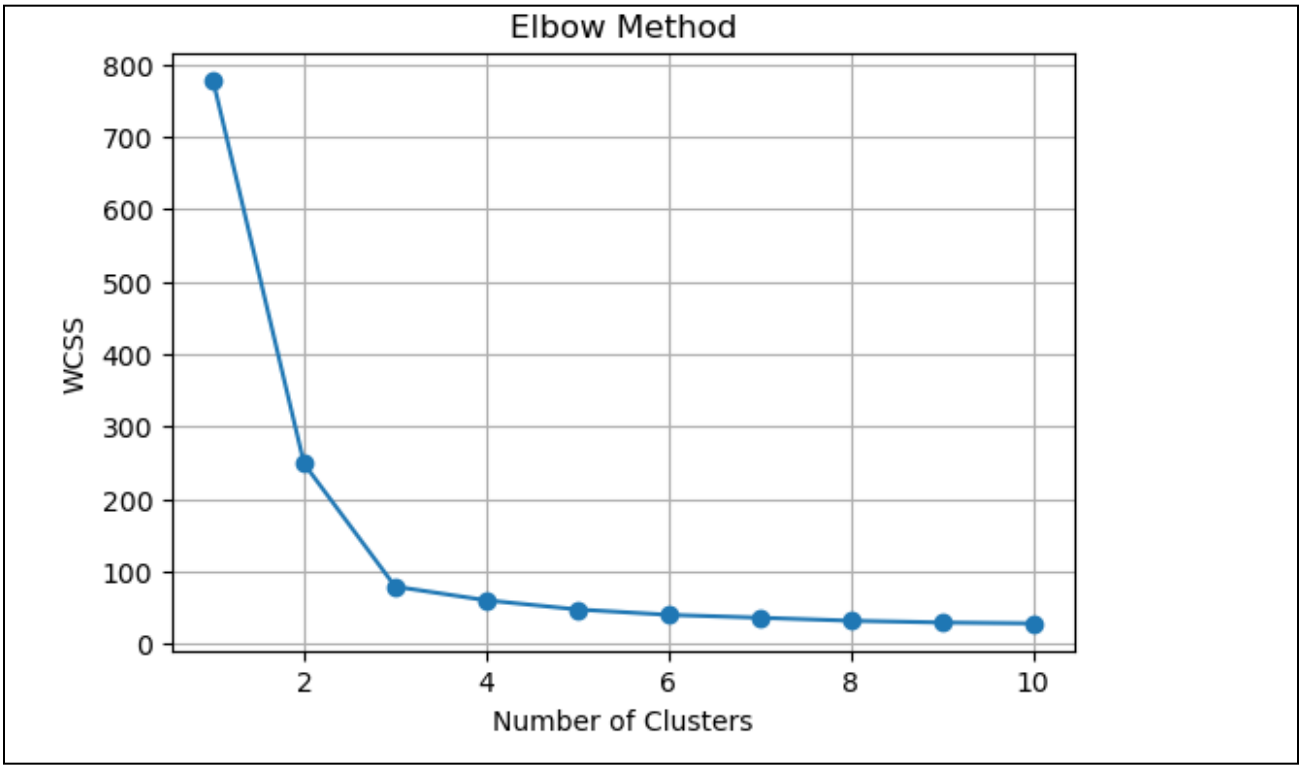
# Check missing values
print(data.isnull().sum())
```

```
Dataset Shape: (150, 4)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 4 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
dtypes: float64(4)
memory usage: 4.8 KB
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
dtype: int64
```

```
# WCSS is square of sum of Distance between data point and centroid.
wcss = []

for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(data)
    wcss.append(kmeans.inertia_)

# Plot Elbow Method graph
plt.figure(figsize=(6,4))
plt.plot(range(1,11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.grid(True)
plt.show()
```




```
# Create KMeans model
kmeans = KMeans(n_clusters=3, random_state=42)
# Predict clusters
data["Cluster"] = kmeans.fit_predict(data)
# Display first rows
data.head()
```

```
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
\
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

   Cluster
0        1
1        1
2        1
3        1
4        1
```

```python
# Print cluster centers
print("Cluster Centers:")
print(kmeans.cluster_centers_)

# Plot clusters using first two features
plt.figure(figsize=(6,5))

# Plot data points
plt.scatter(data.iloc[:,0],
            data.iloc[:,1],
            c=data["Cluster"],
            cmap='viridis',
            s=50)

# Plot cluster centroids
plt.scatter(kmeans.cluster_centers_[:,0],
            kmeans.cluster_centers_[:,1],
            s=250,
            c='red',
            marker='X',
            label='Centroids')

# Labels and title
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("K-Means Clustering with Centroids")

# Show legend
plt.legend()
plt.show()
```

```
Cluster Centers:
[[ 5.06315789e+00  3.44210526e+00  1.50000000e+00  2.73684211e-01
   1.00000000e+00]
 [ 6.61724138e+00  3.05172414e+00  5.48275862e+00  2.04137931e+00
  -8.88178420e-16]
 [ 6.36250000e+00  2.95625000e+00  4.54375000e+00  1.41250000e+00
   2.00000000e+00]
 [ 5.00000000e+00  2.30000000e+00  3.27500000e+00  1.02500000e+00
   2.00000000e+00]
 [ 7.54000000e+00  3.15000000e+00  6.39000000e+00  2.09000000e+00
   0.00000000e+00]
 [ 5.61666667e+00  2.84166667e+00  4.28333333e+00  1.36666667e+00
   2.00000000e+00]
 [ 5.40000000e+00  3.89230769e+00  1.50769231e+00  2.69230769e-01
   1.00000000e+00]
 [ 6.01176471e+00  2.71176471e+00  4.94705882e+00  1.79411765e+00
   2.00000000e+00]
 [ 4.66111111e+00  3.07777778e+00  1.38888889e+00  2.00000000e-01
   1.00000000e+00]
 [ 5.62500000e+00  2.54166667e+00  3.86666667e+00  1.15833333e+00
   2.00000000e+00]]
```

